Lab2 实验报告

实验要求

要求 CPU 支持内存读写。

CPU 需要支持以下指令并通过测试：

- `ld`, `lb`, `lh`, `lw`, `lbu`, `lhu`, `lwu`
- `sd`, `sb`, `sh`, `sw`
- `lui`

实验过程

这一小节中，所有 AI 帮助完成的部分都进行了标注

参考

- AI（DeepSeek, ifow(GLM-4.7)）

Step 0

完成了 `is_stage` 的扩展。由于之前的代码中，要求 `rs1` 必须使用（硬编码使用信号为可用），因此在处理 `lui` 时，将 `rs1` 硬编码为 `0x0`，通过加法产生一个立即数。这也是 RISC-V 的标准操作。

Step 1

完成了 `mem_stage` 阶段的扩展。其中，复杂的 `case` 语句由 `AI` 完成。除了 `dreq.addr` 的特殊要求外，本阶段并没有什么特殊之处。

调试阶段

这一小节中，所有 AI 帮助完成的部分都进行了标注

问题 1

无法正确提交 `store` 类指令。这是因为原本的提交逻辑有误，必须在 `reg_write` 为真时，即发生修改寄存器操作时才提交。而 `store` 指令不写寄存器。

解决 1

修改了提交逻辑，使其在检测到 `pc` 发生变化，且 `inst`（即一路传递的指令）不是空指令时提交。

问题 2

发生了 `load-use` 冒险。因为在 `lab-1` 中，只靠转发就可以解决所有的数据冒险问题，因此没有添加 `stall` 逻辑。

解决 2

为了添加 `stall` 逻辑，对整个流水线进行了多处修改。

首先修改了流水线的时序。在原本的流水线中，有些阶段受到 `step` 信号控制，有些阶段则是组合逻辑，导致整个流水线时序混乱，难以用 `step` 信号整体控制。

同时，由于周期是由 `step` 信号定义的（从一个高值到下一个高值），因此 `stall` 信号的变化也应该持续一个由 `step` 定义的周期，为此，令 `stall = `Enable` 时，置 `fetch_ok = 1`。于是，只要访存阶段完成，`mem_ok = 1`，`stall` 信号就可以被复位了。

最后，假设产生 `stall` 信号时，`if_id_reg` 中的指令是 `p[i]`。`stall` 信号为真时，将 `id_ex_reg` 寄存器清零，`stall` 信号结束后，由于 `step = 1`，会立刻发生一次取指，并跳转到指令`p[i+1]`，于是指令 `p[i]` 被忽略，为了解决这个问题，在产生 `stall` 信号时，首先将 `pc` 置为 `prev_pc`，让 `pc` 重新取到指令 `p[i]`。

以上解决方案中，AI 只给出了实现阻塞机制时，需要实现的效果，代码实现、细节处理都没有 AI 参与

~~实际上 AI 还是参与了的，但是产生的代码质量不高，因此最后决定完全自己写~~

以下是测试结果：

Lab2

```
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x000000008001fffc
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 5,354 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 32,767
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 6,120,097 instr/s
```

Lab1

```
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x0000000080010004
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 2,529 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 16,385
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 6,478,845 instr/s
```